Index

Lab No.	Title	Date of Experiment	Date of Submission	Signature
1)	Bisection Method	2080-09-02	2080-09-05	
2)	Newton Raphson Method	2080-09-05	2080-09-06	
3)	Lagrange's Interpolation	2080-09-06	2080-10-16	
4)	Newton divided difference Interpolation	2080-10-16	2080-11-14	2080/12/8
5)	Introduction to integration and Differentiation	2080-11-14	2080-11-22	
6)	Introduction to Gauss Elimination Method	2080-11-22	2080-12-08	

Lab-1

Write an algorithm and program in C to find the root of the equation $x^2-4x-10$ correct upto 3 decimal places using bisection method

Algorithm

Start

Choose x_l and x_u as two guesses for the root such that $f(x_l)\,f(x_u)<0$ and stopping criterion ε

Compute $f(x_l)$ and $f(x_u)$

Estimate the root, x_m of the equation $f(x)=0$ as the mid-point between x_l and x_u as

$$x_m = \frac{x_l + x_u}{2}$$

Now check the following

a. If $f(x_l)\,f(x_m)=0$; then the root is x_m. Go to step 8

b. Else If $f(x_l)\,f(x_m)<0$, the root lies between x_l and x_m. Set $x_u = x_m$.

c. Else if $f(x_l)\,f(x_m)>0$, the root lies between x_m and x_u. Set $x_l = x_m$

Find the absolute approximate relative error as

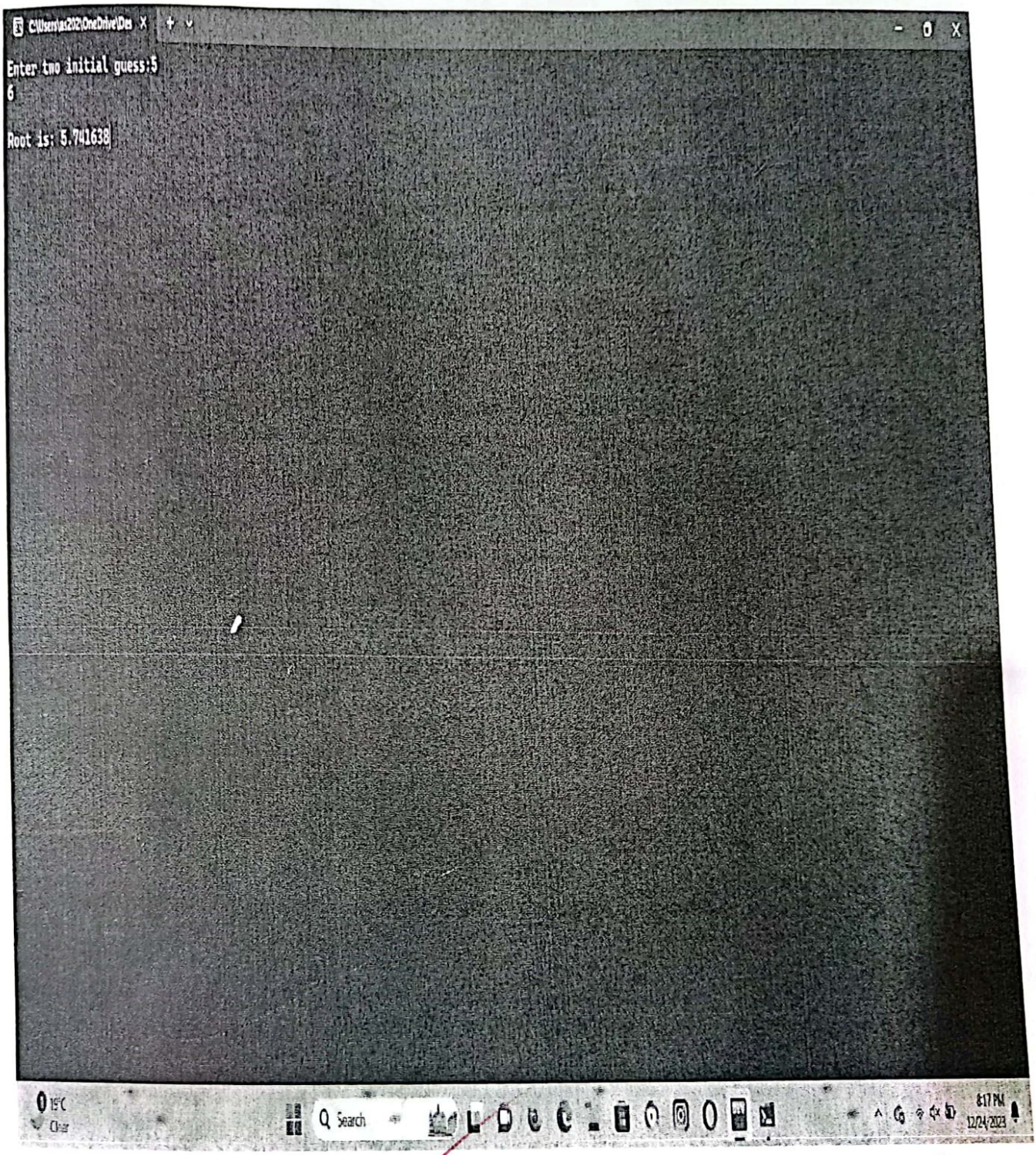
$$|E_{ra}| = \left|\frac{x_1 - x_u}{x_u}\right|$$

If $|E_{ra}|>\varepsilon$, then go to step 3, else go to step 8.

Stop

Code:

```
#include <stdio.h>
#include <conio.h>
#include <Math.h>
#define f(x) x*x-4*x-10
#define E 0.0001
void main()
{
 float x1, x2, x0, f1, f2, f0, err;
 do
 {
  printf("enter two initial guess:");
  Scanf("%f%f", &x1, &x2);
  }while (f(x1) * f(x2)>0);
  do
  {
  x0=(x1 * x2)/2;
  f1 = f(x1);
  f2 = f(x2);
  f0 = f(x0);
  err = fabs (x1-x0);
  If( f1 * f0<0)
   x2 = x0;
   else
   x = x0;
  } while (err> E);
  printf("the root is %f", x0);
  getch();
  }
```


C:\Users\as202\OneDrive\Des
Enter two initial guess:5
6
Root is: 5.741638
15°C
Clear
Search
8:17 PM
12/24/2023

Lab-2

Write an algorithm and program in C to find the root of the equation $x^2-4x-10$ using Newton Raphson Method.

Algorithm

1. Start
2. Input initial guess x_0 and precision E
3. Evaluate $f'(x)$ symbolically
4. use x_0 to estimate the new value of the root x_1 as

$$x_i = x_0 - \frac{f(x_0)}{f'(x_0)}$$

5. find the absolute relative approximate error, $|\epsilon_a|$ as

$$|\epsilon_a| = \left|\frac{x_1 - x_0}{x_1}\right|$$

6. compare the absolute relative approximate error, $|\epsilon_a|$ with the pre-specified relative error tolerance, E.

   If $|\epsilon_a| > E$)

   $x_0 = x_1$

   go to step 3,

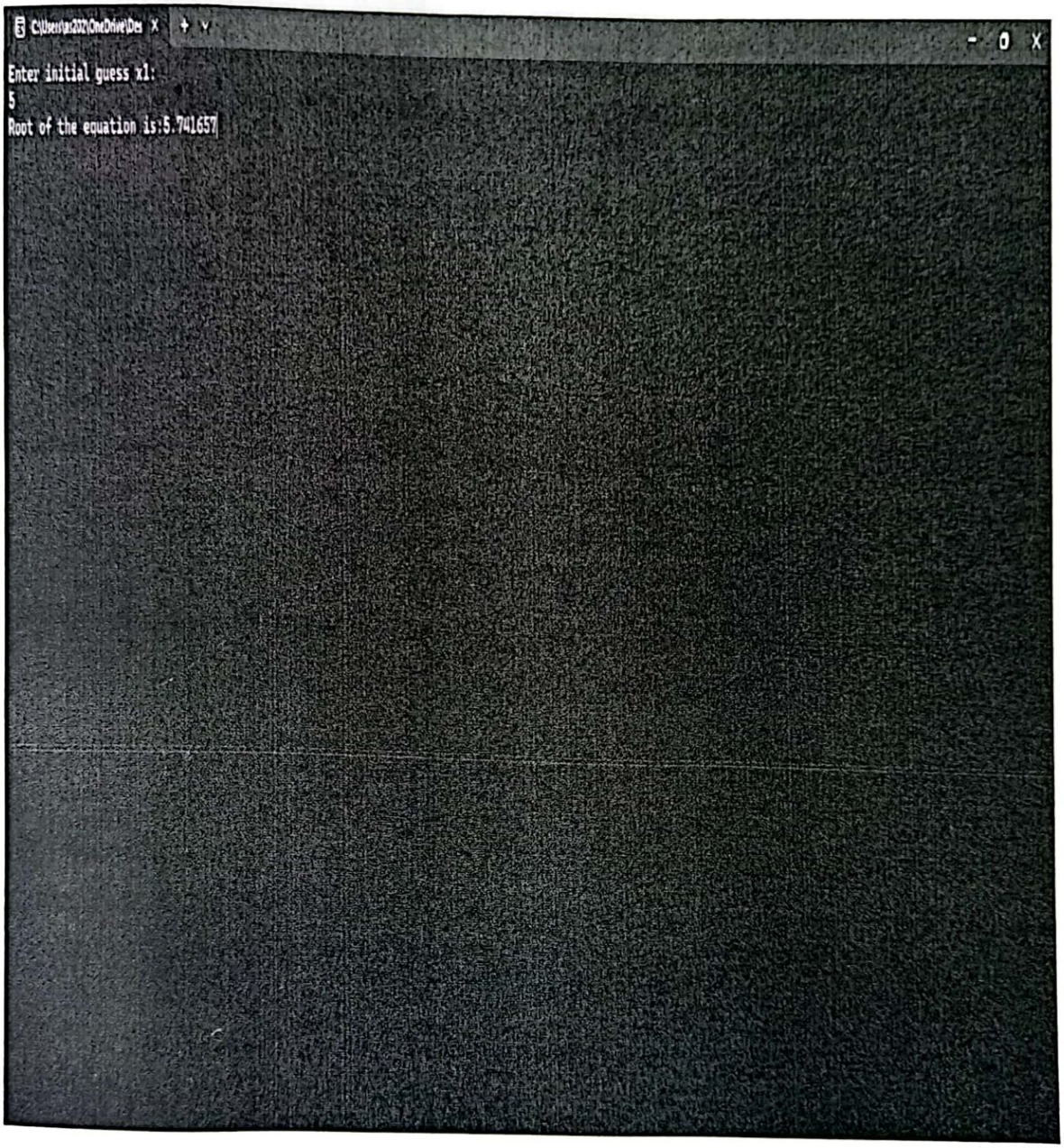
   else

   go to step 7

7. Terminate

Code:

```
#include <stdio.h>
#include <conio.h>
#include <Math.h>
#define f(x) x*x-4x-10
#define fd(x) 2*x-4
#define E 0.0001
void main()
{
   clrscr();
   float x1, x2, f1, f2, err;
   printf("Enter initial guess x1: \n");
   scanf("%f", &x1);
   do
    {
     f1 = f(x1);
     f2 = fd(x1);
     x2 = x1-(f1/f2);
     err = fabs(x2-x1);
     x1 = x2;
   } while(err>E);
   printf("root of the equation is", x2);
   getch();
   }
```



Algorithm:-

Start
Read number of data (n)
Read data x_i and y_i for $i = 1$ to n.
Read value of independent variables say x_p whose corresponding value of dependent say y_p is to be determined.
Initialize: $y_p = 0$
for $i = 1$ to n
set $p = 1$
for $j = 1$ to n
If $i \neq j$ then
calculate $p = (p * (x_p - x_j) / (x_i - x_j))$
End If
Next j
calculate $y_p = y_p + p * y_i$
next i
Display value of y_p as interpolated value.
Stop

Lab 8: Lagrange Interpolation

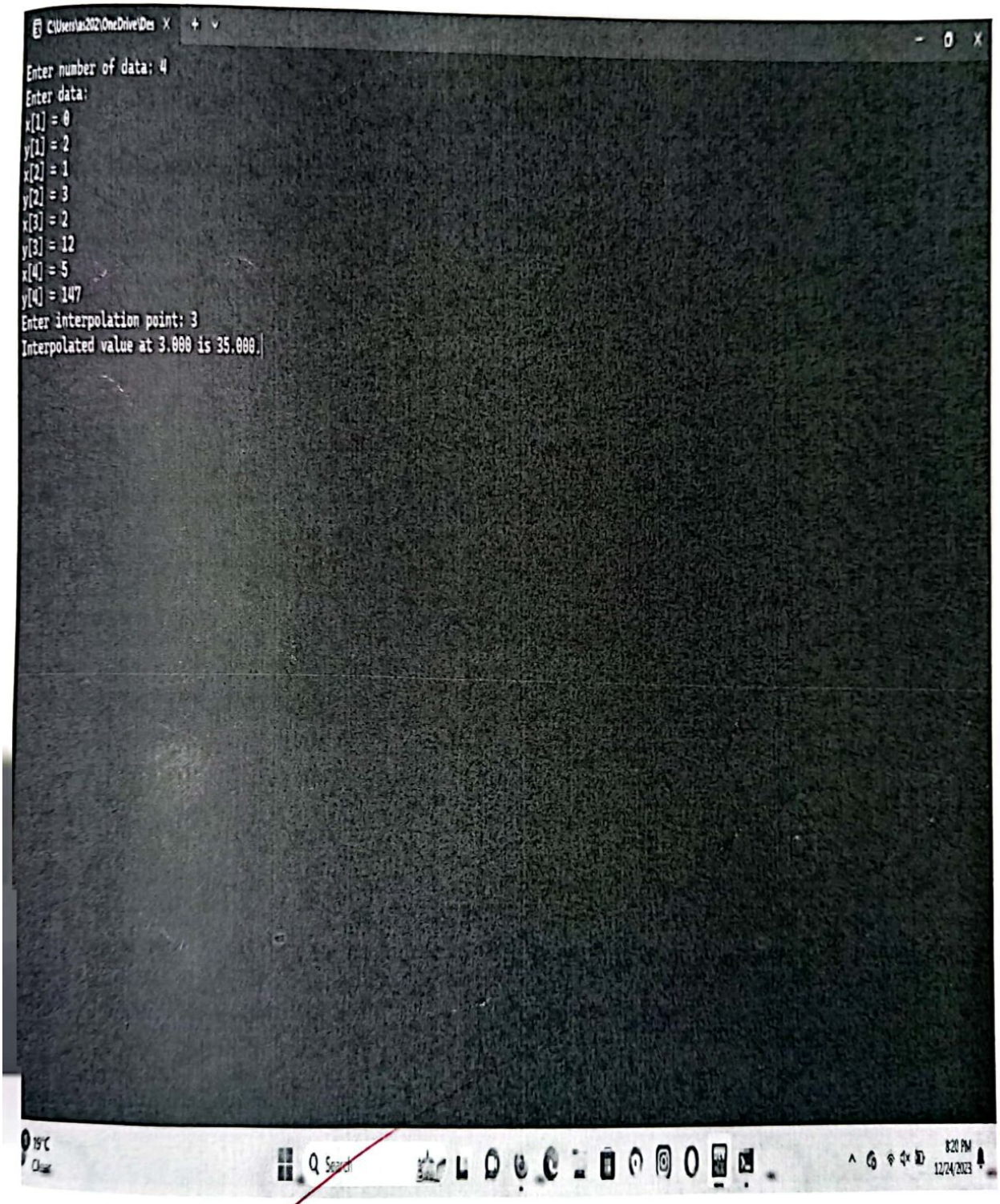
WAP in C to find the value of f at x=3 using lagrange interpolation from table below:

x:	0	1	2	5
f(x):	2	3	12	147

code:

```
#include<stdio.h>
#include<conio.h>
#include<math.h>
void main()
{
 float x[100], y[100], xp, yp=0, p;
 int i, j, n;
 printf("enter number of data:");
 scanf("%d", &n);
 printf("Enter data:\n");
 for(i=1; i<=n ; i++) {
       printf("x[%d]=", i);
       scanf("%f", &x[i]);
       printf("y[%d]=", i);
       scanf("%f", &y[i]);
      }
 printf("enter interpolation point:");
 scanf("%f", &xp);
 for(i=1; i<=n; i++)
  {
    p=1;
    for(j=1; j<=n; j++)
    {
       if(i!=j)
       {
         p=p*(xp-x[j])/(x[i]-x[j]);
       }
```

```
    yp = yp + p * y[i];
}
printf("Interpolated value at %.3f is %.3f.", xp, yp);
getch();
}
```


C:\Users\as2021\OneDrive\Des
Enter number of data: 4
Enter data:
x[1] = 0
y[1] = 2
x[2] = 1
y[2] = 3
x[3] = 2
y[3] = 12
x[4] = 5
y[4] = 147
Enter interpolation point: 3
Interpolated value at 3.000 is 35.000.
8:20 PM
12/24/2023

Lab-4

Write a program to show the Newton's divided difference.

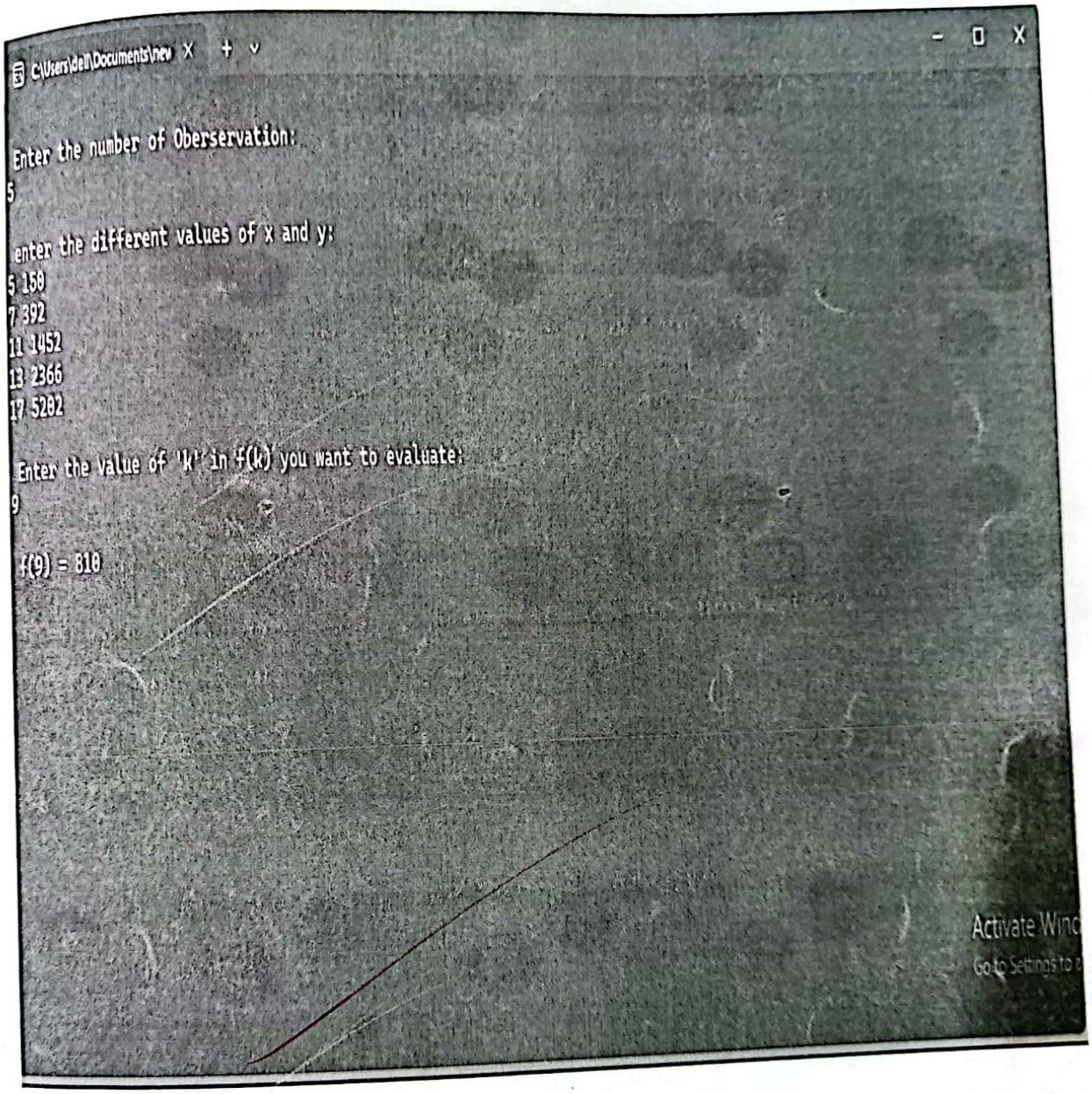
Algorithm

Start
Read number of points, say n
Read the value at which interpolated value is needed, say x
Read given data points
Calculate first divided differences as
for i=0 to n-1
dd[i]=fx[i]
End for
Calculate second to nth divided differences as
for i=0 to n-1
for j=n-1 to i+1
$$dd[j] = \frac{dd[j] - dd[j-1]}{x[j] - x[j-1-i]}$$
End for
End for
Set v=0 and p=1
calculated interpolated value as
for i=0 to n-1
for j=0 to i-1
p=p*(x-xj)
End for
v= v+ dd[i] * p
Reset p=1
End for
print the interpolated value v
Terminate

Code:

```
#include <stdio.h>
#include <conio.h>
void main(){
  int x[10], y[10], p[10];
  int k, f, n, i, j=1, f1=1, f2=0;
  printf("Enter the number of observation:\n");
  scanf("%d", &n);
  printf("\n enter the different values of x:\n");
  for(i=1; i<=n; i++){
    scanf("%d %d", &x[i], &y[i]);
  }
  f=y[1];
  printf("\n Enter the value of 'k' in f(k) you want to evalua
        te:\n");
  scanf("%d", &k);
  do{
    for(i=1; i<=n; i++){
      p[i]=((y[i+1]-y[i])/(x[i+j]-x[i]));
      y[i]=p[i];
    }
  f1=1;
  for(i=1; i<=j; i++){
    f1*=(k-x[i]);
  }
  f2+=(y[1]*f1);
    n--;
    j++;
  }
  while(n!=1);
  f+=f2;
  printf("\n f(%d)=%d", k, f);
  getch();
}
```

lanrange Interpolation Output



lab-5

Introduction to integration and differentiation

WAP in C to evaluate $\int_0^1 \frac{dx}{x+1}$ using

Trapezoidal

Simpson's $\frac{1}{3}$rd

Simpson's $\frac{3}{8}$

assume n=6

Trapezoidal Rule:-

Algorithm

Start

Define function f(x)

Read lower limit of integration, upper limit of integration and number of sub interval

Calculate: step size= (Upper limit - lower limit) / number of sub interval

Set: Integration value = f(lower limit) + f(upper limit)

Set i=1

If i > number of sub interval then go to

Calculate: k= lower limit + i*h

Calculate: Integration value= Integration value + 2* f(k)

Increment i by 1 i.e. i=i+1 and go to step 7

Calculate: Integration value= Integration value * step size/2

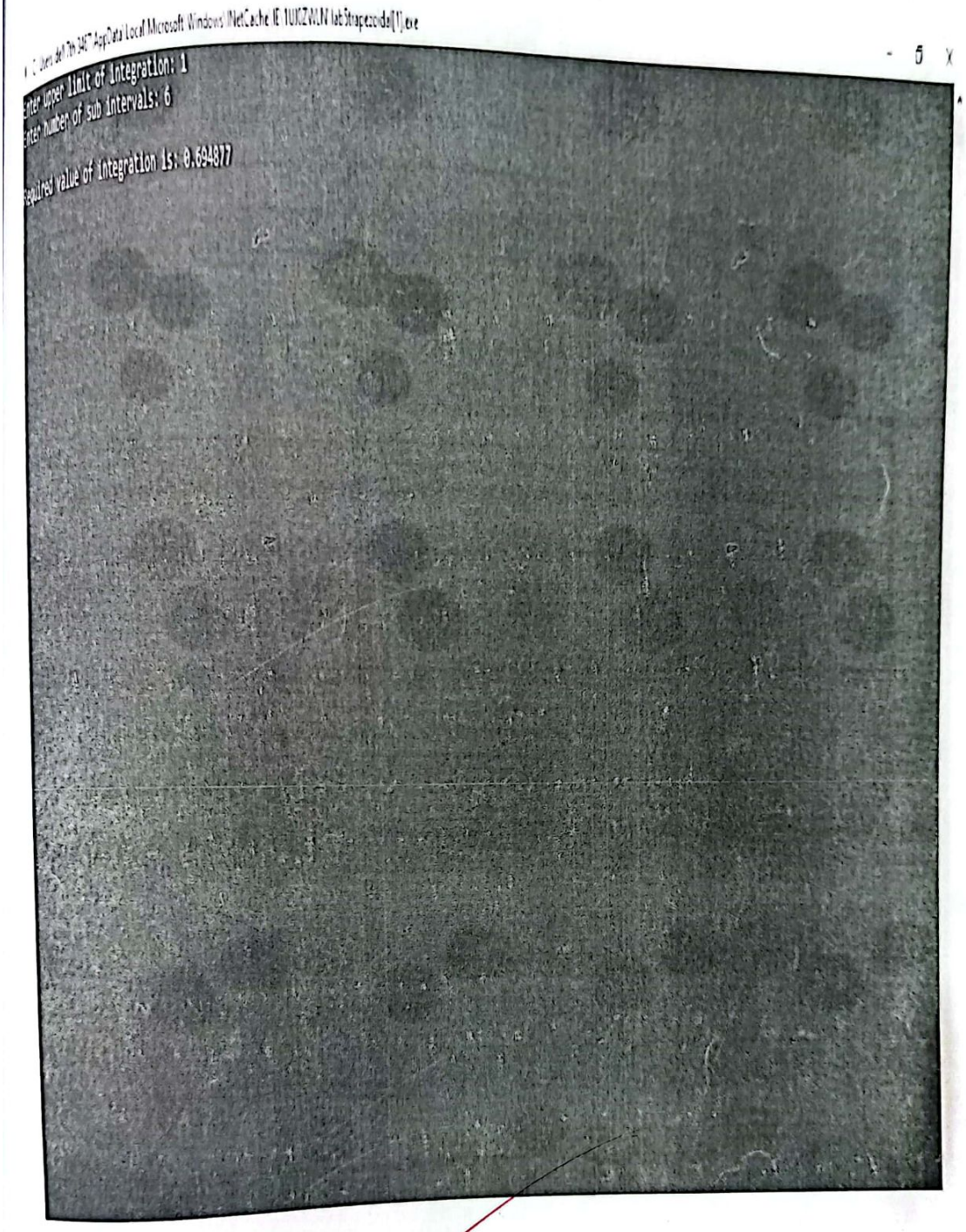
Display Integration value as required answer

Stop.

Trapezoidal

code:

```
#include <stdio.h>
#include <conio.h>
#include <math.h>
#define f(x) 1/(x+1)
int main()
  {
   float lower, upper, integration=0.0, stepsize, k;
   int i, subInterval;
   printf("Enter lower limit of integration:");
   scanf("%f", &lower);
   printf("Enter upper limit of integration:");
   scanf("%f", &upper);
   printf("Enter number of sub intervals:");
   scanf("%d", &subInterval);
   stepsize=(upper-lower)/subInterval;
   integration=f(lower)+f(upper);
   for(i=1; i<=subInterval-1; i++)
   {
    k=lower+i*stepsize;
    integration=integration+2*f(k);
   }
   integration=integration*stepsize/2;
   printf("\n Required value of integration is: %f",
     integration);
   getch();
   return 0;
   }
```


C:\Users\dell\7th SEM" AppData\Local\Microsoft\Windows\INetCache\IE\1UKZWLN\lab3trapezoidal[1].exe
Enter upper limit of integration: 1
Enter number of sub intervals: 6
Required value of integration is: 0.694877

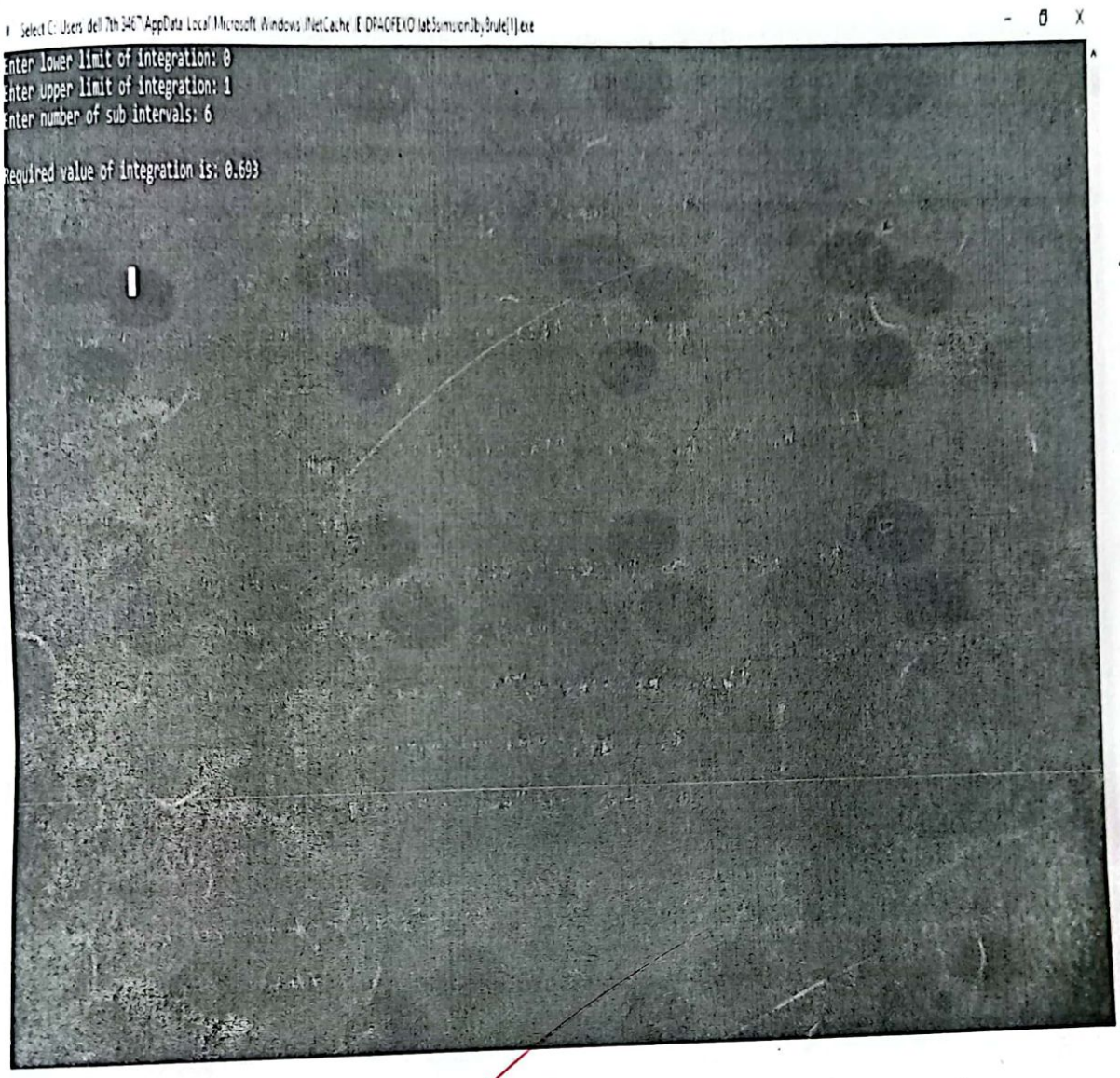
Simpson's $\frac{1}{3}$rd

Algorithm

start
Define function f(x)
Read lower limit of integration, upper limit of integration and number of sub interval
calculate: step size = (upper limit - lower limit) / number of sub interval
set: integration value = f(lower limit) + f(upper limit)
set i = 1
If i > number of sub interval then goto
calculate: k = lower limit + i*h
If i mod 2 = 0 then
Integration value = Integration value + 2 * f(k)
otherwise
Integration value = Integration value + 4 * f(k)
End If
Increment i by 1 i.e. i = i+1 and go to step 7.
calculate: Integration value = Integration value * stepsize/3
Display Integration value as required answer
Stop

Code: (Simpson's $\frac{1}{3}$rd)

```
#include <stdio.h>
#include <conio.h>
#include <Math.h>
#define f(x) 1/(x+1)
int main(){
 float lower, upper, integration=0.0, stepsize, k;
 int i, subInterval;
 printf("Enter lower limit of integration:");
 scanf("%f", &lower);
 printf("Enter upper limit of integration:");
 scanf("%f", &upper);
 printf("Enter number of sub intervals:");
 scanf("%d", &subInterval);
 stepsize=(upper-lower)/subInterval;
 integration=f(lower)+f(upper);
 for(i=1; i<=subInterval-1; i++)
  {
   k=lower+i*stepsize;
   if(i%2==0)
   {
    integration=integration+4*f(k);
   }
  }
integration=integration*stepsize/3;
printf("\n Required value of integration is: %.3f",
        integration);
getch();
return 0;
}
```


Select C:\Users\dell 7th 340\AppData\Local\Microsoft\Windows\INetCache\IE\DP4QFEXO\lab3simpson3by3rule[1].exe
Enter lower limit of integration: 0
Enter upper limit of integration: 1
Enter number of sub intervals: 6
Required value of integration is: 0.693

Simpson's $\frac{3}{8}$ rule

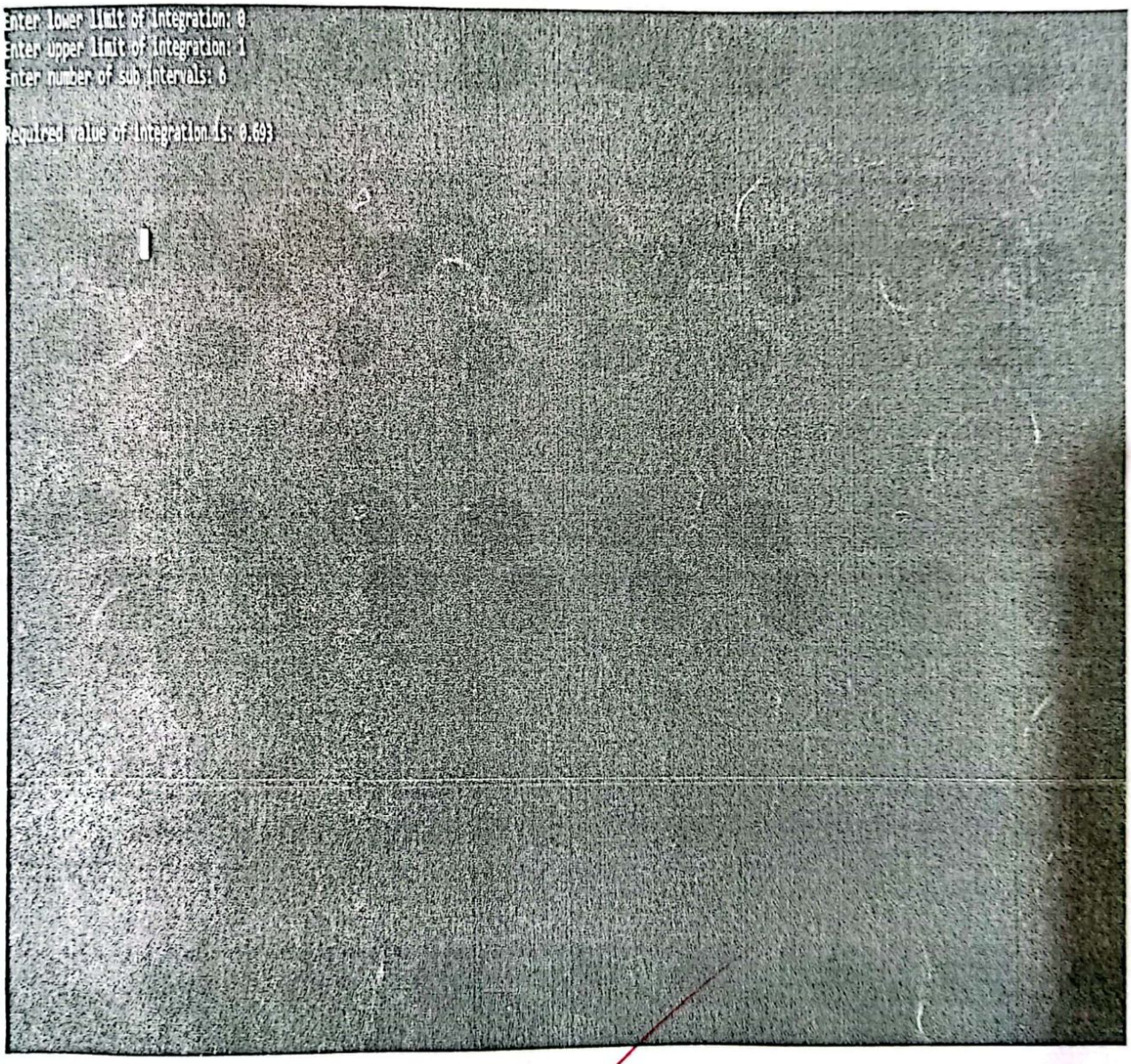
Algorithm

Start
Define function f(x)
Read lower limit of integration, upper limit of integrati
and number of sub interval.
calculate: step size = (upper limit - lower limit)/number of
sub interval
set: integration value = f(lower limit) + f(upper limit)
set: i = 1
If i > number of sub interval then goto
calculate: k = lower limit + i*h
If i mod 3 = 0 then
Integration value = Integration value + 2*f(k)
Otherwise
Integration value = Integration value + 3*f(k)
End If
Increment i by 1 i.e. i = i+1 and go to step 7.
calculate: Integration value = Integration value
step size * $\frac{3}{8}$
Display Integration value as required answer.
stop.

Simpson's 3/8

Code:

```
#include <stdio.h>
#include <conio.h>
#include <Math.h>
#define f(x) 1/(x+1)
int main()
 {
 float lower, upper, integration=0.0, stepsize, k;
 int i, subinterval;
 printf("Enter lower limit of integration: ");
  scanf("%f", &lower);
  printf("Enter upper limit of integration: ");
  scanf("%d", &upper);
  printf("Enter number of sub intervals: ");
  scanf("%d", &subinterval);
  stepsize = (upper-lower)/subinterval;
   integration f(lower)+f(upper);
   for(i=1; i<=subinterval-1; i++)
     {
    k=lower + i*stepsize;
    if(i%3==0)
    {
    integration=integration + 2* f(k);
    }
    else {
     integration= integration + 3*f(k);
    }
   }
  integration= integration * stepsize *3/8;
  printf("\n Required value of integration is: %3f",
       integration);
   getch();
   return 0;
    }
```


Enter lower limit of integration: 0
Enter upper limit of integration: 1
Enter number of sub intervals: 6
Required value of integration is: 0.693

lab 6: Introduction to Gauss Elimination Method

AP in C to solve the following equation using Gauss imination Method.

```
#include <stdio.h>
#include <conio.h>
#include <math.h>
#include <stdlib.h>

#define SIZE 10

int main()
{
float a[SIZE][SIZE], x[SIZE], ratio;
int i,j,k,n;

printf("Enter number of unknowns:");
scanf("%d", &n);

for(i=1;i<=n;i++)
 {
  for(j=1;j<=n+1;j++)
   {
    printf("a[%d][%d]= ", i, j);
    scanf("%f", &a[i][j]);
   }
 }

 for(i=1;i<=n-1;i++)
  {
   if(a[i][i]==0.0)
    {
     printf("Mathematical Error!");
     exit(0);
    }
    for(j=i+1;j<=n;j++)
     {
      ratio=a[j][i]/a[i][i];
      for(k=1;k<=n+1;k++)
       {
        a[j][k]= a[j][k] - ratio *a[i][k];
       }
```

```
    }
}
x[n]= a[n][n+1]/a[n][n];
for(i=n-1; i>=1; i--)
{
  x[i]= a[i][n+1];
  for (j=i+1; j<=n; j++)
  {
     x[i]= x[i] - a[i][j]*x[j];
  }
  x[i]= x[i]/ a[i][i];
  printf("\n solution: \n");
  for(i=1; i<=n; i++)
  {
    printf("x[%d]= %0.3f\n", i, x[i]);
  }
  getch();
  return(0);
}
```

2080/1/26